Final Project
BitTorrent, A peer to peer file sharing system

Release Date 1405/04/23
Deadline 1405/05/15

Introduction

In this course project you will implement the famous bittorent protocol developed by "Bram Cohen"[1]. Bittorent is a distributed and peer to peer file sharing protocol. It was developed in order to mitigate the shortages of normal client-server communication. For example, in a scenario where the demand for a certain file is rather high, the content delivery network (which is made of a number of servers) may fail due to congestion and servers may crash. Also there is a single point of failure, meaning there is a single hot spot which delivers the files and in the event of failure, the whole system would collapse and there is no other way around it. Bittorent protocol suggests that instead of downloading in a server-client fashion, the peers download from each other. With the advent of broadband technologies such as DSL and cable modems, the everyday user like yourself suddenly has a big chunk of bandwidth, not only for download, but also upload. Sharing files directly from your computer (without first sending them to a server) is now a reality. A peer downloading a certain file may simultaneously upload this file to the other peers downloading. This ecosystem helps utilizing the bandwidth of the network and the amazing thing, is that when the number of downloading peers increase, the rate of download also increases. Try to figure out why! This makes the system behavior a little counter-intuitive and exciting.

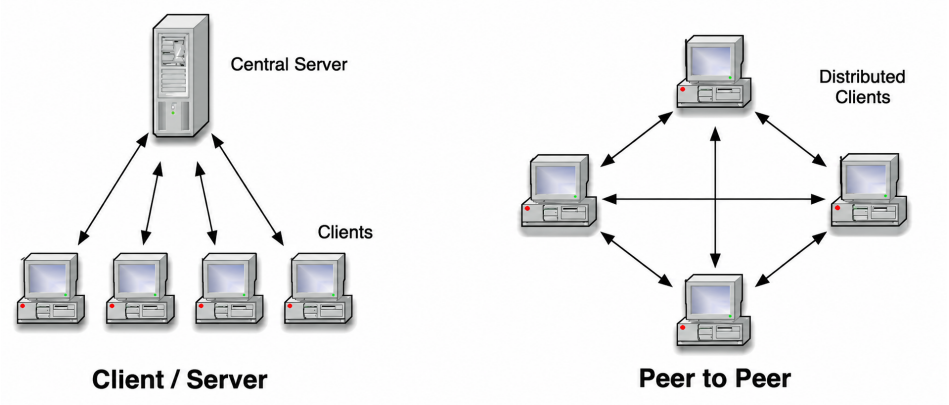

Figure 1: p2p vs client-server

Warning

[1] https://en.wikipedia.org/wiki/Bram_Cohen

Bittorent protocol is a beautiful protocol designed for decent purposes but unfortunately it is misused by the majority of people in order to bypass copyright rules and download illegal contents. This project does not motivate students to use torrenting. But a little search about common ways of torrenting, available client implementations, and powerful search engines would not harm anyone and might be useful for your understanding of how torrenting works. For more information about Torrenting safety, read articles published in this regard[a].

[a] For example visit *Are torrents safe in 2020?*, *What Is BitTorrent and Is It Safe?*

Implementation

Please note:

- You must only use Python language.
- You are allowed to use available Bencoding libraries and tools only for making HTTP requests. These libraries must not be used to implement or simplify any part of the BitTorrent/Bencoding protocol or other networking logic. In your report, you must explicitly state all external libraries used.
- Any innovation in design would be more than welcome! Sky is the limit! You can add any other features to your program, but remember the ones mentioned in this document are necessary.
- Many important features of bittorent are not necessary in this project; for instance you don't need to worry about port forwarding, NAT, firewalls, etc. since peers and trackers are implemented on your localhost or on different VMs which are in the same IP range.

Getting Started

You can study the complete Bittorent specification here[2].

System Model

For pretty much all p2p systems most people follow these basic steps:

1. Search for something. (Bittorent doesn't have this feature built-in. You have to find the torrents yourself, probably with Google or some other popular search engine.)
2. Get a list of everyone who is sharing what they want.
3. Go through the list and ask each person on it if they will please send the file.
4. Once the file is complete, start sharing it with other people in the system.

The overview of the system is depicted in Figure 2. The bittorent ecosystem is comprised of the following entities:

1. Indexer (search engine)

[2] `http://bittorrent.org/beps/bep_0003.html`

2. Tracker Server
3. Bittorent Client

We will discuss each entity in the upcoming sections.

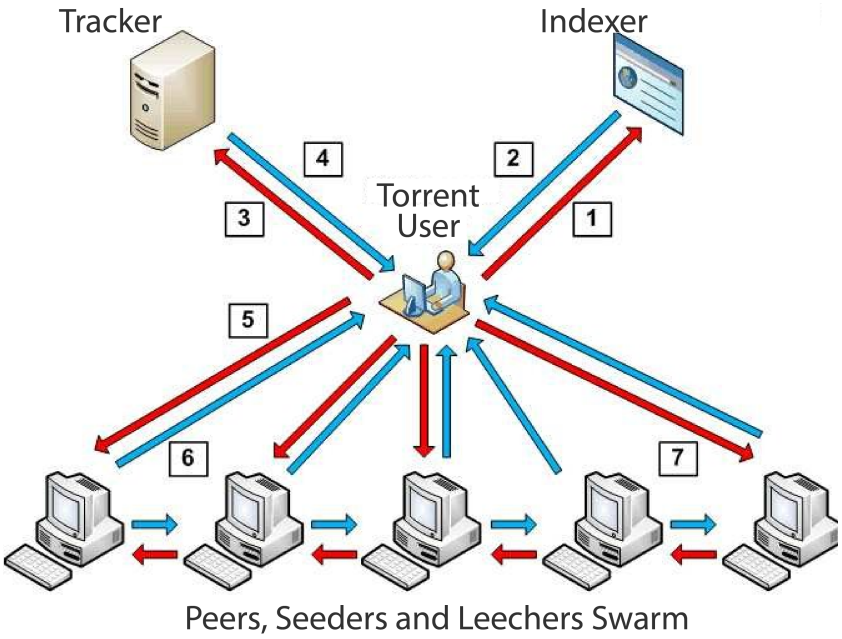

Figure 2: Bittorent Architecture

Indexer

An "indexer" is a site that compiles a list of torrents and descriptions and is a place where users form a community (with rules!) around BitTorrent content. When you want to share, download, or request files, the indexer's community is where you go. These usually take the form of a forum and/or an IRC channel.

In this project you don't need to build an indexer. You need to prepare at least 2 `.torrent` files. These bencoded files have the same content as a normal `.torrent` file but you may implement them in a different format like a simple `.txt`.

These files will be altered in the grading phase as a test case so make them human readable and try not to use Binary or Hex formats. The complete file specification will be discussed in the following sections.

Tracker

The tracker is an HTTP/HTTPS service which responds to HTTP GET requests. A tracker keeps track of people who are interested in torrents. When you download a `.torrent` file it contains a link to a tracker as well as an identifier (hash) which is unique to that specific torrent. Your BitTorrent client then connects to the tracker and asks for a list of all people interested in that torrent. At the same time, the tracker adds you to that list so that other people know that you are interested. Your BitTorrent client will also periodically ask the tracker for an updated list. That's all a tracker does: keep track of that list for each torrent, and give it out to people who are interested. The tracker does not know anything else about the torrent, nor does it send you the file. It just shows you where to go to get the file.

Peer

The peers get the peer lists from the tracker server and start sharing (Download & Upload) the files using TCP connection with other peers. At a high level, every peer is required to register with the tracker in order to get a list of peers. The tracker response includes a dictionary of peers, as well as a timeout value during which the peer is expected to re-ping the tracker for an updated peer list. This pinging also serves the purpose of notifying the tracker that the peer is still up and running, and if the peer does not respond within the timeout period, the tracker assumes that the peer has died and removes it from its own peer list.

Once peers are connected, they can be either interested or not interested in each other's data. If they are interested, then they send an `interested` message to the other peer. The peer can then decide whether to *choke* or *unchoke* them. If they decide to choke them, then they refuse to upload to them for now. They may choose to unchoke them later. When a peer decides to unchoke another peer, then it notifies the other peer. That peer can now start requesting data and the peer should respond with the requested data. These relationships are bidirectional, one for uploading and one for downloading, but each direction is independent of the other.

Implementation

These are the entities that you **must** implement in your code:

Metainfo File

The MetainfoFile (aka the `.torrent` file), contains all the information about your torrent, including where the Tracker URL is, what the piece-length, piece-count, and all the piece SHA-1's are, and a bunch of other meta-data. If you don't know what a "Hash Function" is, check this link[3]. There is absolutely no need to implement the hash function! You may use any library that suits you. A torrent file is a bencoded (again, you may use external libraries for bencoding) dictionary with the following keys:

1. `announce`: A list of Tracker Server URLs (in this project you only need to give the IP address of the tracker server you are going to implement).
2. `info`: this maps to a dictionary whose keys are dependent on whether one or more files are being shared:
   (a) `files`: a list of dictionaries each corresponding to a file (only when multiple files are being shared). Each dictionary has the following keys:
      i. `length`: size of the file in bytes.
      ii. `path`: a list of strings corresponding to subdirectory names, the last of which is the actual file name (in this project you only need the name; there is no sub-directory).
   (b) `length`: size of the file in bytes (only when one file is being shared).
   (c) `name`: suggested filename where the file is to be saved (if one file) / suggested directory name where the files are to be saved (if multiple files).
   (d) `piece length`: number of bytes per piece. This is commonly 2^8 KiB $= 256$ KiB.
   (e) `pieces`: a hash list, i.e., a concatenation of each piece's SHA-1 hash. As SHA-1 returns a 160-bit hash, `pieces` will be a string whose length is a multiple of 20 bytes. If the torrent contains multiple files, the pieces are formed by concatenating the files in the order they appear in the `files` dictionary.

[3] `https://en.wikipedia.org/wiki/SHA-1`

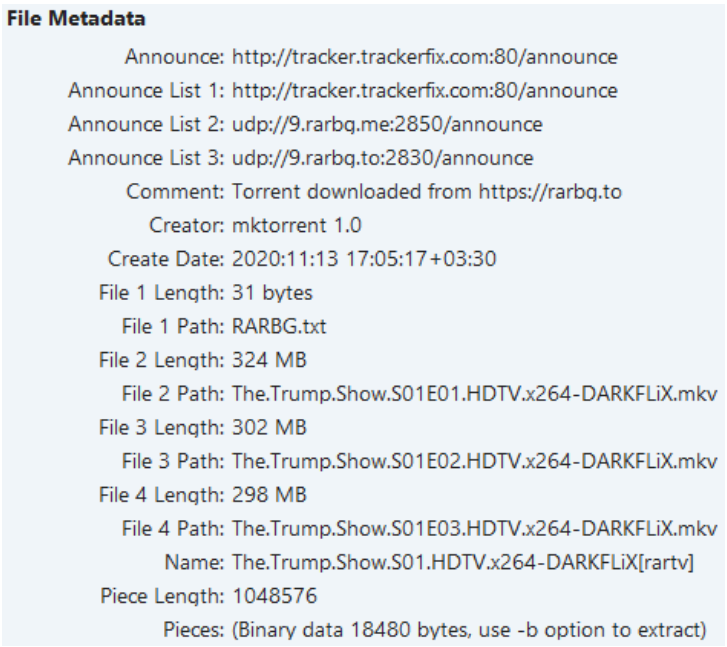

Figure 3: Sample .torrent file metadata

A single-file de-bencoded Metainfo sample:

```
{
    'announce': 'http://bttracker.debian.org:6969/announce',
    'info':
    {
        'length': 678301696,
        'name': 'debian-503-amd64-CD-1.iso',
        'piece length': 262144,
        'pieces': <binary SHA1 hashes>
    }
}
```

A multiple-file de-bencoded Metainfo sample:

```
{
    'announce': 'http://tracker.site1.com/announce',
    'info':
    {
        'files':
        [
            {'length': 111, 'path': ['111.txt']},
            {'length': 222, 'path': ['222.txt']}
        ],
        'name': 'directoryName',
        'piece length': 262144,
        'pieces': <binary SHA1 hashes>
    }
}
```

Tracker Web Service

You need to implement a simple Tracker web service that responds to HTTP requests. The web service should be implemented with multi-threading techniques[4]. The Tracker base URL consists of the "announce URL" (which is an IP address in our case) as defined in the metainfo (`.torrent`) file. The parameters are then added to this URL, using standard CGI methods (i.e. a '`?`' after the announce URL, followed by '`param=value`' sequences separated by '`&`').

Tracker Request Parameters

The parameters used in the client → tracker GET request are as follows:

- `info_hash`: URL-encoded 20-byte SHA1 hash of the value of the `info` key from the Metainfo file.
- `peer_id`: URL-encoded 20-byte string used as a unique ID for the client, generated by the client at startup. This is allowed to be any value.
- `port`: The port number that the client is listening on. Ports reserved for BitTorrent are typically 6881–6889.
- `uploaded`: The total amount of bytes uploaded (since the client sent the `started` event to the tracker) in base ten ASCII.
- `downloaded`: The total amount of bytes downloaded (since the client sent the `started` event to the tracker) in base ten ASCII.
- `left`: The number of bytes this client still has to download in base ten ASCII.
- `event`: If specified, must be one of `started`, `completed`, `stopped`, (or empty which is the same as not being specified). If not specified, then this request is one performed at regular intervals.
  - `started`: The first request to the tracker must include the event key with this value.
  - `stopped`: Must be sent to the tracker if the client is shutting down gracefully.
  - `completed`: Must be sent to the tracker when the download completes.

Tracker Response Parameters

The tracker responds with a `text/plain` document consisting of a bencoded dictionary with the following keys:

- `failure reason`: If present, then no other keys may be present. The value is a human-readable error message as to why the request failed (string). This failure could be because of rapid requests the client sent or any other issue.
- `interval`: Interval in seconds that the client should wait between sending regular requests to the tracker.
- `complete`: Number of peers with the entire file, i.e. seeders (integer).

[4] `https://mimo.org/glossary/python/multithreading`

- `incomplete`: Number of non-seeder peers, aka "leechers" (integer).
- `peers` (dictionary model): The value is a list of dictionaries, each with the following keys:
  - `peer_id`: peer's self-selected ID, as described above for the tracker request (string).
  - `ip`: peer's IP address IPv4 (dotted quad).
  - `port`: peer's port number (integer).

Peer

Peer code is completely separated from Tracker web service code. You should design a peer with the following abilities:

- **request**: Sends the aforementioned requests to the tracker.
- **Response Handler**: Receives the aforementioned response from the tracker and parses the data.
- **Parse**: Parses the Metainfo file.
- **Torrent Thread Pool**: Creates threads for each of the different torrent files.
- **Ping Peers**: Pings all other peers on a regular basis. (This part is to make sure that the end-to-end process works fine.)

Logger

For a better debugging procedure and clarification you must implement a logger that logs each and every event both on the peer side and the tracker side. The logger minimum architecture:

- **Header**
  - File name
  - Creation Date/Time
  - Last Modified Date/Time
- **Body**
  - Event Type
  - Date/Time
  - Description

Wireshark and Packet Observation

Pieces vs. Network Packets

Be careful not to confuse BitTorrent pieces with TCP/IP packets. A *piece* is an application-level part of the shared file. Its size is defined by the `piece length` field in the Metainfo file, and its correctness can be checked using the SHA-1 hashes stored in the `pieces` field. This concept is part of the BitTorrent protocol and can be tested deterministically.

A network packet, on the other hand, is created and managed by the operating system and the network stack. One application-level message may be split into multiple TCP packets, and multiple small messages may also be combined into fewer packets. This behavior may change depending on the operating system, network configuration, buffering, timing, and other environmental details.
Therefore, the grading pipeline will focus on application-level correctness, such as valid Metainfo parsing, correct piece count, correct piece hashes, valid tracker registration, and correct peer lists. The exact number or boundary of TCP packets observed in Wireshark will not be used as a strict grading criterion.

Using Wireshark for Debugging

Wireshark is a very useful tool for observing the actual packets exchanged between your peers and tracker. You are not required to implement anything specifically for Wireshark, but you are strongly encouraged to use it while debugging your project. In your report, you may include Wireshark screenshots or packet capture explanations showing:

- A screenshot of HTTP GET requests from peers to the tracker.
- Tracker request parameters such as `info_hash`, `peer_id`, `port`, and `event`.
- A screenshot of HTTP responses sent by the tracker.
- A *Follow TCP Stream* view, or an equivalent explanation, for peer-to-peer communication.
- The ports used by the tracker and peer services.
- A short explanation that TCP packet boundaries are not used as the correctness criterion.

This part is mainly intended to help you understand and verify your own implementation. Providing clear Wireshark evidence in the report may be considered as a bonus, especially if it helps explain how your system behaves during execution.

Submission

Please submit **all required materials** as a single `.zip` archive. The archive must be named as follows:

`DN_FinalProject_<StudentID>.zip`

Replace `<StudentID>` with your own student ID. The ZIP archive must include all the items listed below.

1. A folder for the Tracker code and tracker log files.
2. A folder for the Peer code, peer log files, and at least two Metainfo files.
3. A complete `report.pdf` file explaining what you did from scratch.